

Memory In Long Video Generation

- Autoregressive vs Diffusion Transformer
- Hybrid Models
- KV cache vs Long horizon memory
- Memory storage choices
- Subset retrieval architectures
- Proposed architecture for memory under a fixed budget

Autoregressive vs DiT

- **Pure Autoregressive**

- $p(V) = \prod p(x_t | x_{<t})$
- Frame X_N depends on frames $X_{0:N-1}$
- No noise. Simply next token
- Only next frame prediction

- **Diffusion Transformer**

- $p(\hat{V}) \sim p_\theta(V | I, C)$
- Given
 - conditioning image
 - Caption
- Generate a whole video



Autoregressive Discrete Diffusion

- This is **LUMOS**. It is a hybrid approach
- It is also autoregressive, and it does next frame prediction
- But tokens within each frames are iteratively demasked
 - Let tokens $M_t[1 - 1000]$ represent ONE frame X_t
 - Then $X_{t+1} = M_{t+1}[1 - 1000]$, but to generate this:
 - Initially $M_{t+1}[1 - 1000] = [\text{MASK}, \text{MASK}, \text{MASK}, \dots \text{MASK}]$
 - Parallel unmasking using all $M_{0:t}$ [Using all previous tokens]
 - Finally $M_{t+1}[1 - 1000]$ becomes [Tree, person, road, etc...]

Autoregressive Diffusion For Long Video Generation

- **Hybrid Approach Which Is Most Papers**
- Generate a chunk just like a standard DiT
- Use the whole past to generate the new chunk
- **Next chunk prediction**
- $\hat{V}[t + 1:t + H] \sim p_{\theta}(V_{t+1:t+H} | V_{0:t}, I_t, C)$
- Given condition frame I, caption C, and past chunks, generate next chunk of size H
- ***We can't choose all past chunks forever***

KV Cache vs Memory

- KV Cache

- Used for a single chunk generation (around 1-5 seconds at a time)
- More appropriate for pure autoregressive models
- Stores attention K/V tokens
- Short-lived: current context window
- Goal: faster inference / avoid re-computation

- Long-Horizon Memory

- Stores scene/history information
- Long-lived: beyond context window
- Goal: prevent forgetting and preserve consistency

- Key Difference

- KV cache solves compute.
- Memory solves forgetting.

Long Video Generation Objective

$$\bullet \hat{V}[t+1:t+S] \sim P_{\theta}(V_{t+1:t+S} | I_t, M_t, c,)$$

Generated Video from time-step $t+1$ to $t+S$ where S is the chunk size

Ground truth video

Conditioning Image \ last generated frame

Memory representation of the whole past

Caption

Memory Representation

- Criteria?
 - Compact
 - Informative
- Memory Representation
 - RGB Frames → Context-as-Memory, CovRAG
 - Latent Frames → WorldMem, LongLive-RAG
 - Surfels → VMem
 - 3D Point Clouds → Memory Forcing
 - TSDF Voxels → SPMem
 - KV Tokens → SlotMemory
 - StreamVGGT Features → Captain Safari
- Retrieval Metadata
 - Camera Pose → Almost everyone
 - Timestamp → WorldMem
 - FoV / Intrinsic → VMem, CovRAG, Safari
 - Semantic Slots → SlotMemory

Retrieval Mechanisms

- FOV / View-Overlap Retrieval→ Context-as-Memory, WorldMem
- Pose-Conditioned Retrieval→ Captain Safari, WorldMem
- Target-View Coverage Retrieval→ CovRAG
- Residual Coverage / Non-Redundant Retrieval→ CovRAG
- Surfel Visibility Retrieval→ VMem

Retrieval Mechanisms

- Point-to-Frame Geometry Retrieval→ Memory Forcing
- Rendered Spatial-Memory Conditioning→ SPMem
- Latent Similarity Retrieval→ LongLive-RAG
- Semantic Slot / KV Relevance Retrieval→ SlotMemory
- Time / Recency Retrieval→ WorldMem, LongLive-RAG, many baseline settings
- Learned Retriever / Learned Memory Query→ Captain Safari, SlotMemory, LongLive-RAG

The Literature Gap

- Most studies focus on the type of memory to save and the retrieval strategy
- Underlying assumption?
- That we have infinite memory bank
- So I did a memory complexity analysis on existing methods

Memory Complexity

Method	Storage unit	RAM formula (outdoor)	Complexity	@ 10 min	@ 60 min
Context-as-Memory	Raw latent frames	$N \times 512 \text{ KB}$	$O(N)$	8.8 GB	53 GB
WorldMem	Raw latent frames + pose	$N \times 512 \text{ KB} + N \times 0.1 \text{ KB (pose)}$	$O(N)$	8.8 GB	53 GB
Memory Forcing	Dual raw latent banks	$2 \times N \times 512 \text{ KB}$	$O(N)$	17.6 GB	105 GB
VMem (open-world)	Surfels (4.1 KB each)	$N \times 4000 \times 4.1 \text{ KB}$	$O(S)$, $S \sim O(N)$ open-world	281.5 GB	1689 GB
MosaicMem	3D patch tokens + episodic frames	$N \times 256 \times 4.1 \text{ KB} + E \times 512 \text{ KB}$	$O(N)$ dual overhead	18.1 GB	108 GB
SPMem	Point cloud + capped episodic	$P(t) \times (12+d) \times 4B + \min(E, N) \times 512 \text{ KB}$	$O(P) + O(E)$, P grows sub-linearly	1976.1 GB	1976 GB

Proposed Idea BENCHMARK EVICTION POLICY

- Under a fixed budget of memory size K :
 - Test long video generation quality
 - Test different eviction policy [Similarity, FOV overlap, pose-similarity]
 - Adapt retrieval strategies as eviction policy
 - Test my approach [Utility conditioned memory pruning for long video generation]
 - This is also a very useful benchmark. What eviction policy does how well on long video generation when memory is fixed and small.
 - It's not a subset selection problem, but a subset retention problem

Formal Problem Statement

- Given memory bank $M_S = \{m_1, m_2, m_3, \dots, m_S\}$, total memory
- Let m_{S+1} be the new memory coming to the bank
- What memory item should be evicted among them?
- Baselines:
 - FIFO
 - Least Recently Used
 - Non-Max Suppression (nearest neighbor eviction)

CVPR 2026 Trends from Visual Geometry Group

CVPR26: Meta Insights into Trends and Tendencies (37/152)

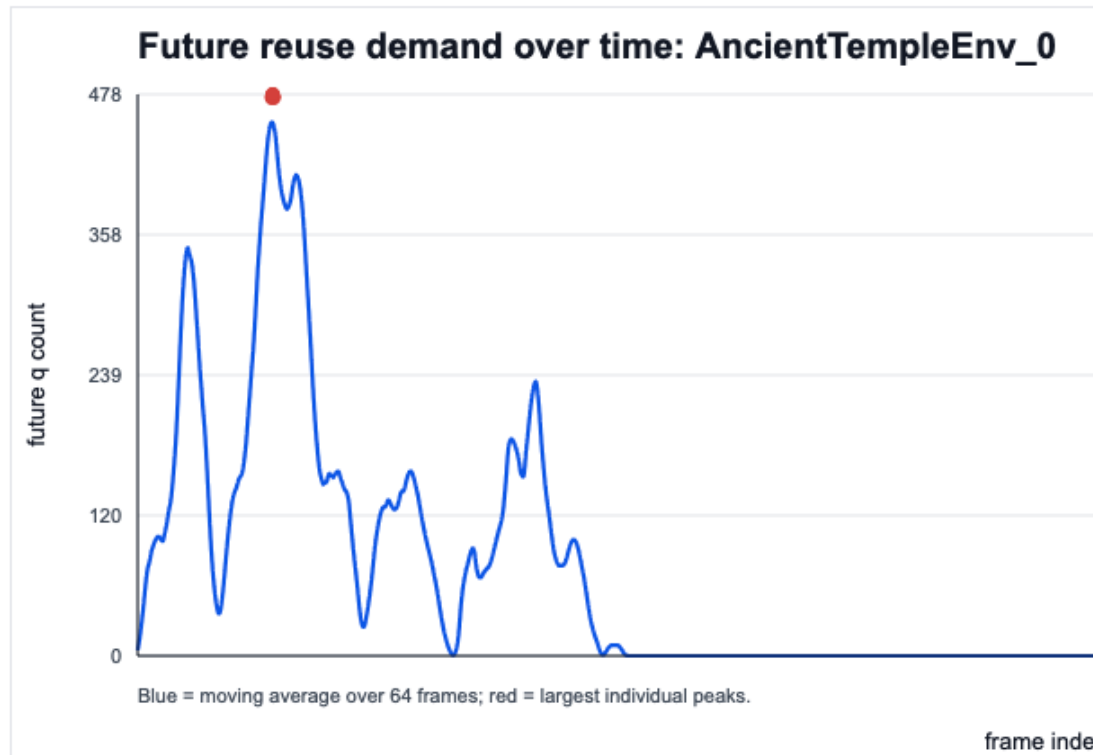
Robotics related workshop: meta insights

- ❑ Robotics is becoming a full-stack physical AI problem: data, models, sensors, simulators, deployment, and reliability must scale together.
 - ❑ Data: cross-embodiment robot data, human videos, simulation, generated trajectories, autonomous rollouts
 - ❑ Models: VLA, world-action models, diffusion policies, hierarchy, memory, promptable policies
 - ❑ Embodiment: tactile sensing, contact-rich manipulation, dexterous hands, hardware/edge deployment
 - ❑ Systems: modular pipelines, sim-to-real, deployment flywheel, reliability, multi-robot / robot web

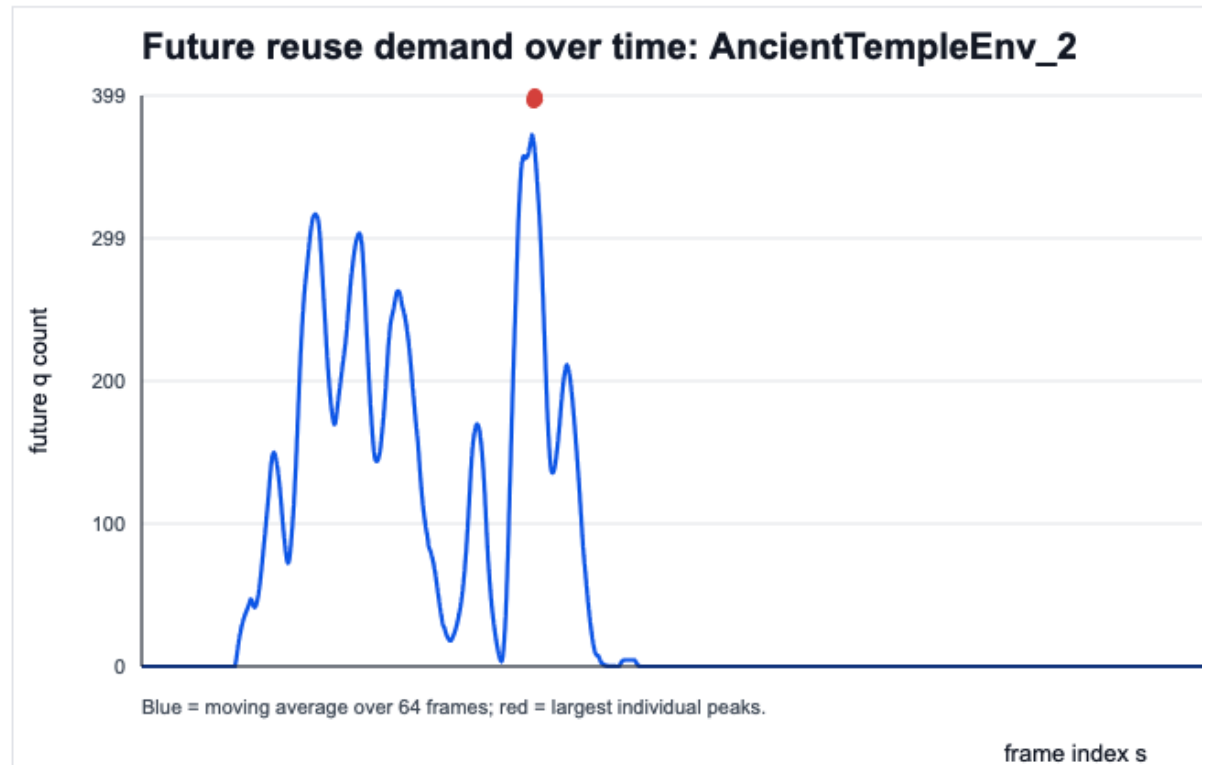
- ❑ Future directions
 - ❑ From VLA to world-action model
 - ❑ Human videos to robotics, alignment is the key (action alignment, sensor alignment, structured representation alignment)
 - ❑ From synthetic scenes to real-video-derived interactive worlds
 - ❑ Tactile, force, contact-rich: for dexterous robotics
 - ❑ Long-horizon/longer-horizon tasks require hierarchy + memory + promptability
 - ❑ From benchmark success to production metric
 - ❑ Towards scalable physical AI (now already), which could also be extended to scalability in deployment

What frames were actually useful?

future reuse timeline AncientTempleEnv 0

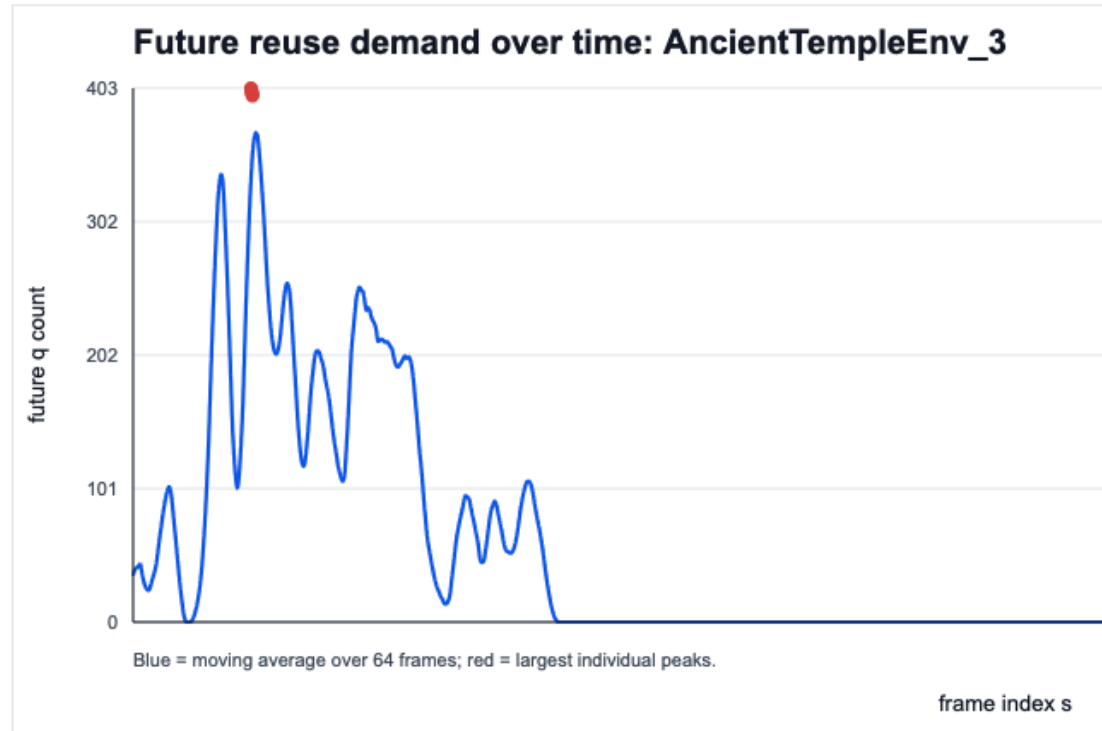


future reuse timeline AncientTempleEnv 2

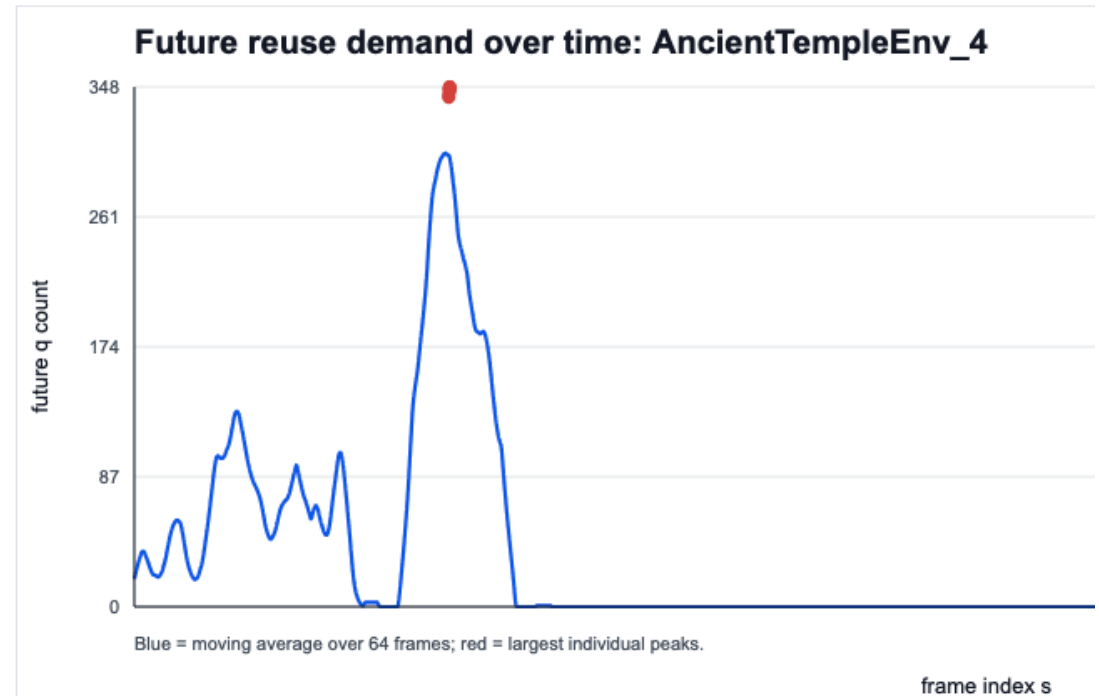


What frames were actually useful?

future reuse timeline AncientTempleEnv 3



future reuse timeline AncientTempleEnv 4



A stat Fact

- Overlap demand is sparse and highly concentrated
- Across 5,000 target frames, only 20.1% of past frames are reused at all
- while the top 5% of frames account for 50.7% of all future reuse.

How Far Back Do You Need To See?

Past Window W	Useful Past Links Covered	Queries Fully Covered
120 frames	13.4%	4.9%
256 frames	28.1%	13.8%
512 frames	47.1%	30.2%
1024 frames	74.0%	54.7%
2048 frames	99.9%	98.9%

Utility Conditioned Memory Pruning For Long Video Generation

- Transform memory bank to form k clusters
- Define rarity
- M_S is the memory bank
- where $n(m_i)$ is the size of cluster m_i belongs to
- Define $Sim(m_i|M_S) = \arg \min_{m_j \in M_S, i \neq j} D(m_i, m_j)$
- $D(m_i, m_j) = D_{img}(I_i, I_j) + \lambda D_{pose}(Pose_i, Pose_j)$
- $Utility(m_i) = Rarity(m_i|M_S) \times Sim(m_i, M_S)$

Deliverables

- I will show usefulness of my memory pruning technique
- I will include Pack Cache style of memory optimization
- I will introduce all the famous KV cache compression mechanism as a baseline
- I will introduce traditional baseline
- Finally will report video generation quality, 3d consistency, memory consistency metrics

UPDATE JUNE 19

- Recap:
- We are generating long horizon videos (10-60 seconds or more)
- Long horizon video generation requires past frames to be saved
- All memory papers achieving good long horizon generation
 - Studies the memory representation (what to save)
 - Studies how to retrieve the best subset from the memory bank (what to retrieve)
- I am studying what happens when the memory has a small budget
- Nobody actually did this analysis yet
- Which past frames matters for long video generation (research question)

Update June 19

- Long video generation's top 3 most useful downstream tasks
- Input: Image + camera trajectory -> Video following camera
- Input: Image + actions -> Video following actions
- Input: Image + only caption -> Video following caption narrative

Task 1: Camera Trajectory

- Given $C_{1:T}, I_0, p$
 - Where $C_{1:T}$ is the entire input camera trajectory (positions, orientations)
 - I_0 is the initial image
 - p is the caption
 - Generate $V_{1:T}$
- Let video generated by model up to time t be $V_{1:t}$
- Let memory stored by model up to time t be M_S (at full capacity)
- We want to keep generating chunks
- $V_{t+1:t+H} = G\theta(I_t, C_{t+1:t+H}, \text{subset}(M_S))$

Task 1: Details

- Dataset: Context-as-memory [Kling team]
- Backbone: MemCam (wan-2.1)
- Durations: 10s, 20s, 40s, 60s
- Memory Budget: 16, 32, 64, 128 items
- 15 scenes, each with their unique trajectory
- First policies to check:
 - Unbounded (upper bound, no limit on memory)
 - FIFO
 - Looking for more baseline policies
 - My idea (Rarity x Irreplaceability)

Task 1: Preliminary Results

- Duration 10s
- Budget 32 frames
- Metrics DINO distance, LPIPS, FVD

run	DINO distance ↓	LPIPS ↓	FVD ↓
Upper Bound	0.3176	0.5112	578.09
FIFO	0.3356	0.5342	580.84
Ours	0.3237	0.5071	557.86

Examples



Infinite memory



Sliding Window



Our Method

Task 2: Plans



- WorldMem Minecraft dataset
- WorldMem World Model
- NTU CV lab

Task 2: Action Trajectory

- Given $A_{1:T}, I_0, p$
 - Where $A_{1:T}$ is the entire input actions(front back left right jump)
 - I_0 is the initial image
 - p is the caption
 - Generate $V_{1:T}$
- Let video generated by model up to time t be $V_{1:t}$
- Let memory stored by model up to time t be M_S (at full capacity)
- We want to keep generating chunks
- $V_{t+1:t+H} = G\theta(I_t, A_{t+1:t+H}, subset(M_S))$

